

AI SQL Studio

Complete Website Feature Showcase

A polished product document covering the implemented website features, user journeys, backend capabilities, security controls, database models, API surface, and deployment readiness.

6

AI SQL TOOLS

3

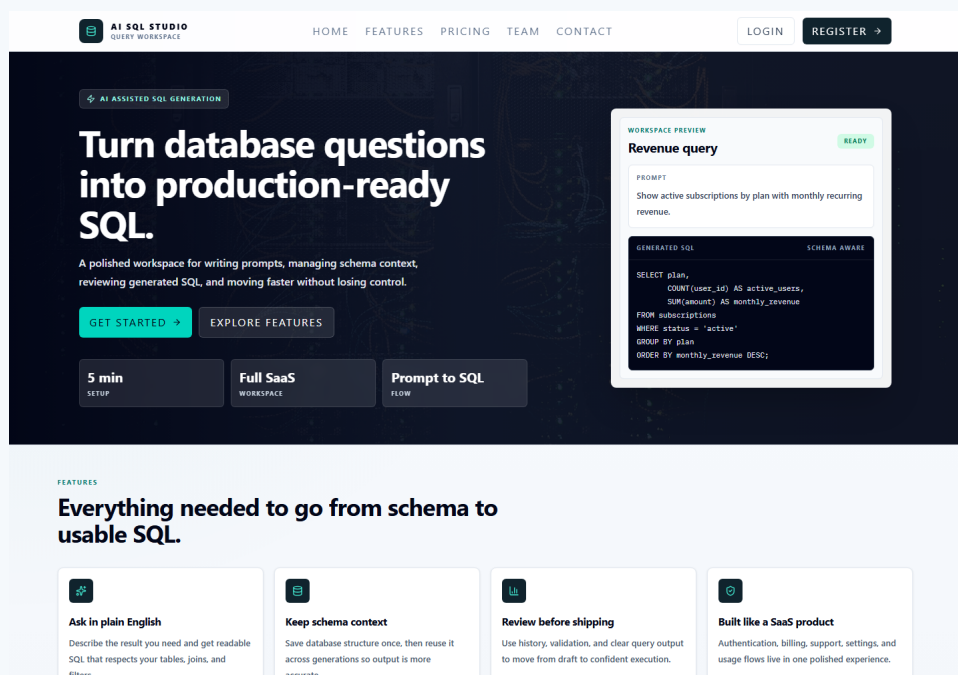
PLANS

12

CORE MODELS

10

API AREAS



Generated: May 31, 2026

Repository documentation for the AI SQL Studio website and platform.

Executive Summary

What the website is

AI SQL Studio is a full-stack SaaS product for generating, improving, validating, formatting, and explaining SQL with AI. It combines a public marketing site, authenticated user dashboard, billing system, analytics, feedback/support surfaces, and an admin control center.

The project is more than a single prompt form. It is implemented as a portfolio-ready web application with React/Vite on the frontend and Express/MongoDB on the backend. The system integrates Clerk for auth, Gemini for AI output, Razorpay for billing, email utilities for transactional workflows, OpenAPI for documentation, and Docker/Render files for deployment.

Product strengths

End-to-end SaaS flow

Public site, auth, protected dashboard, pricing, plan gating, payments, invoices, support, feedback, admin operations, and deployment configuration.

Real AI workflow

Schema-aware prompting, six AI SQL tools, strict JSON AI responses, repair logic, SQL formatting, history persistence, and productivity analytics.

Professional operations

Admin dashboard for user moderation, access requests, feedback triage, notification publishing, security review, revenue, and audit logs.

Security-aware implementation

Clerk sessions, HttpOnly admin cookie, signed payment/webhook checks, CORS/CSRF controls, rate limits, validators, body limits, and env validation.

COMPLETE WEBSITE SCOPE

Feature Inventory

PUBLIC

Public Website and Product Presentation

The public website introduces the product, pricing, workflow, platform preview, contact path, login/register entry points, billing redirects, and a developers showcase.

Implemented in this project

- Routes: /, /developers, /login, /register, /billing/success, /billingsuccess, plus redirects for FAQ, support, feedback, billing, and admin.
- SEO component, robots.txt, sitemap.xml, manifest, favicon, and image assets under docs/screenshots.
- Responsive landing sections with auth modal, plan cards, product preview, workflow storytelling, and developer profiles.

AUTH

Clerk Authentication, Protected Routing, and Workspaces

Clerk owns user login and sessions. The frontend waits for Clerk readiness, while the backend validates Clerk sessions, API keys, account states, and organization context.

Implemented in this project

- ClerkProvider, AuthModal, ClerkTokenBridge, ProtectedRoute, PublicRoute, AdminProtectedRoute, and AdminPublicRoute.
- Backend Clerk middleware, bearer token auth, paid-plan API key verification, workspace role and permission checks.
- Account restrictions for suspended, rejected, pending, and deleted users with an access appeal flow.

AI

AI SQL Workspace

The Generate page supports six AI tools and keeps the workflow compact: choose a mode, provide SQL or a prompt, run the tool, inspect output, copy/download, chain to another tool, or save generated schema.

Implemented in this project

- Modes: generate, schema, optimize, validate, explain, and format.
- Gemini prompting with strict JSON responses, JSON repair, SQL review, SQL completion recovery, SQL formatter fallback, and result persistence.
- Plan-aware tool gating, SQL dialect selection for paid users, schema awareness, keyboard shortcuts, and provider error handling.

SCHEMA

Schema Context

Users can store a database schema per personal or organization workspace, giving AI requests a trusted table and column context.

Implemented in this project

- Schema page, schemaService client, schema routes, schema controller, schema service, and Schema model.
- Workspace scoping for personal vs organization schema context.
- Validation and body limits protect the schema endpoint from oversized payloads.

HISTORY

Query History and SQL Productivity

AI outputs are saved as query records so users can search, filter, copy, export, delete, pin, favorite, tag, and track SQL workflow usage.

Implemented in this project

- Query history page with search, mode filter, sort, pagination, copy, export, and delete.
- Pro tools: pins, favorites, tags, full searchable archive, and productivity tracking.
- Backend query service, query validator rules, workspace scope utilities, request cache, and export helpers.

ANALYTICS

Overview and Advanced Analytics

The dashboard reports plan state, usage, recent activity, quality signals, workflow signals, top tables, time saved, and reliability indicators.

Implemented in this project

- Overview page, Analytics page, Recharts charts, stat cards, free analytics preview, and upgrade prompts.
- Backend aggregations for daily activity, weekly growth, mode mix, optimizer usage, validation changes, and query quality counters.
- DSA utilities for top-K table extraction, query analytics accumulation, pagination, search, and caching.

BILLING

Pricing, Billing, Invoices, and Plan Management

The app includes Free, Professional, and Team plans with Razorpay checkout, payment verification, invoices, downgrade support, and effective plan resolution.

Implemented in this project

- Pricing page, Billing Success page, Invoices page, and productConfig plan data.
- Razorpay order/payment-link creation, signed payment verification, signed webhooks, invoice records, and confirmation emails.
- Plan middleware, subscription service, organization subscriptions, downgrade flows, and subscription cron support.

SUPPORT

Feedback, Support, FAQ, Settings, and Notifications

The authenticated app includes operational pages and user communication flows that make the product feel complete beyond the AI screen.

Implemented in this project

- Support, FAQ, Feedback, Settings, NotificationDrawer, WorkspaceSwitcher, and Clerk account/organization profile embeds.
- Feedback submission and feedback history endpoints.
- Notification create/list/read/read-all flows, notification read tracking, and notification ranking utility.

ADMIN

Admin Console and Platform Operations

A separate admin console provides user management, revenue visibility, feedback triage, notification publishing, security review, access appeals, and audit history.

Implemented in this project

- Admin login with HttpOnly admin cookie, protected admin dashboard, theme toggle, charts, search, filters, and pagers.
- User moderation actions: set Pro, set Free, approve access, reject access, suspend, unsuspend, and delete.
- Admin APIs for overview, users, feedback, notifications, access appeals, security events, and audit/security logging.

API, Security, Tests, and Deployment Readiness

The backend is organized as a production-style service with route/controller/service/model layering, security middleware, validation, OpenAPI docs, tests, Docker, and Render configuration.

Implemented in this project

- Express app with Helmet, CORS allowlist, CSRF origin checks, rate limits, JSON body limits, validators, async error handling, and centralized API responses.
- OpenAPI JSON at `/api/docs/openapi.json` and route groups for auth, AI, schema, queries, payment, feedback, notifications, webhooks, and admin.
- Backend Node tests, frontend Vitest tests, ESLint, Dockerfiles, `nginx.conf`, `docker-compose.yml`, and `render.yaml`.

FRONTEND ROUTES AND EXPERIENCES

User-Facing Application

The frontend uses route-level code splitting and protected routing to separate public pages, user dashboard pages, payment callback pages, and admin pages.

Area	Routes	Purpose
Public	<code>/</code> , <code>/developers</code> , <code>/login</code> , <code>/register</code> , <code>/billing/success</code> , <code>/billingsuccess</code>	Product, auth entry, developer showcase, and payment callback handling.
Dashboard	<code>/dashboard</code> , <code>/generate</code> , <code>/schema</code> , <code>/history</code> , <code>/analytics</code> , <code>/settings</code>	Protected user workspace for AI, schema, archive, analytics, and account settings.
Operations	<code>/dashboard/billing</code> , <code>/invoices</code> , <code>/support</code> , <code>/faq</code> , <code>/feedback</code>	Billing management, invoice history, help content, and feedback submission.
Admin	<code>/admin/login</code> , <code>/admin/dashboard</code>	Separate control center for platform operations and moderation.

Dashboard pages documented

- Overview: plan status, usage, free credit state, recent queries, and quick actions.
- Generate: AI workspace for generation, schema creation, optimization, validation, explanation, and formatting.
- Schema: save, edit, clear, and use schema context for accurate SQL generation.
- History: search, filter, sort, copy, export, delete, pin, favorite, tag, and track SQL actions.
- Analytics: Pro dashboard for usage charts, top tables, workflow metrics, quality score, productivity, and reliability.
- Billing/Pricing: Starter, Professional, and Team plans with checkout and current billing state.
- Invoices: invoice history for personal or organization workspace.
- Settings: Clerk user and organization profile integration.
- Support, FAQ, Feedback: user help, guidance, feedback submission, and previous feedback.

AI Workspace Details

The Generate page is the centerpiece of the product. It connects frontend controls to a backend AI service that normalizes dialects, checks plans, loads workspace schema context, calls Gemini, repairs structured output, formats SQL, persists history, and refunds usage on failures.

Tool	Who can use it	What it does
Generate	Starter and paid	Converts plain English into SQL using saved schema context.
Schema	Paid	Creates relational SQL DDL from a product or domain description.
Optimize	Paid	Rewrites SQL for better performance while preserving semantics.
Validate	Paid	Repairs syntax and obvious logical SQL issues.
Explain	Paid	Produces a concise technical explanation with steps, columns, performance notes, and risks.
Format	Paid	Formats SQL locally with sql-formatter fallback behavior.

AI implementation behaviors

- Schema context is treated as authoritative for generation and review.
- The backend asks Gemini for strict JSON output and then repairs malformed AI responses when needed.
- Generated SQL is reviewed against schema context and completed if the AI provider truncates output.
- Saved queries include prompt, generated SQL, mode, dialect, workspace owner, and timestamps.
- Free users have limited generation credits while paid users unlock advanced modes and dialect selection.
- AI provider errors are converted into helpful product messages for quota, model, auth, timeout, payload size, and bad response states.

Pricing and Plan Access

Plan	Price	Included capability
Starter	INR 0 / month	5 one-time generation credits, 1 schema workspace, text-to-SQL, copy/download SQL, latest 10 history entries, and basic overview.

Professional	INR 499 / month	Advanced AI tools, unlimited app-side usage subject to provider quota, full history, pins, favorites, tags, dialects, analytics, invoices, and priority support.
Team	INR 1499 / month	Everything in Professional plus team workspace creation, shared organization schema, shared organization history, member invitations, and team analytics.

Plan enforcement points

- Frontend locks advanced modes for users without a paid plan and shows upgrade guidance.
- Backend plan middleware protects Pro and Team-only routes.
- Organization workspaces require Team plan access.
- Effective plan resolution supports personal plan, personal Team entitlement, and organization subscription state.
- Billing APIs return current plan, personal plan, workspace plan, renewal data, and billing management permission.

RAZORPAY AND INVOICES

Billing and Payment Flow

Billing is implemented across frontend pages, backend services, Mongoose records, Razorpay integration, and email utilities. The app supports hosted payment links, order verification, payment-link verification, signed webhooks, invoice records, subscription activation, and downgrade.

Checkout creation

Backend creates Razorpay orders or hosted payment links for Pro or Team and records a pending Payment document with user/workspace scope.

Callback verification

Frontend payment success page sends Razorpay callback data to the backend, which checks HMAC signature and local payment ownership.

Webhook processing

Signed Razorpay webhooks process payment_link.paid and payment.captured events and safely ignore unsupported or unmatched events.

Invoice and activation

Successful payments create Payment and Invoice records, activate Pro/Team state, and can send confirmation email with invoice PDF.

PLATFORM OPERATIONS

Admin Console

The admin console is separate from the user dashboard. It provides user management, revenue monitoring, feedback triage, notification preferences, and more.

Admin dashboard capabilities

- Admin authentication with credential validation, rate limiting, and HttpOnly admin cookie.

- Overview stats for users, paid users, free users, total queries, invoices, revenue, feedback, access appeals, notifications, and security events.
- Charts for monthly revenue/signups, feedback status, and plan distribution.
- User search and filters by plan, status, and access status.
- Moderation actions: set Pro, set Free, approve access, reject access, suspend, unsuspend, and delete.
- Notification creation with title, message, type, priority, audience, status, publish date, and expiry date.
- Access appeal workflow with statuses: new, in_review, resolved, and closed.
- Feedback triage by status and search with admin note support.
- Security event review with severity/status labels and risk-focused user lists.
- Admin audit logs and security events are written for sensitive admin actions.

OPENAPI AND ROUTE GROUPS

Backend API Surface

The backend publishes OpenAPI JSON and is organized into focused route modules. Most user routes require Clerk authentication and approved access; admin routes require admin auth.

Group	Routes	Purpose
System	GET /api/health, GET /api/docs/openapi.json	Health checks and API contract.
Auth	GET /api/auth/me, POST /api/auth/logout, POST /api/auth/access-appeal	Current profile, logout, and access appeals.
AI	POST /api/ai	Generate, schema, optimize, validate, explain, and format SQL.
Schema	GET/POST/DELETE /api/schema, POST/DELETE /api/schema/clear	Workspace schema context lifecycle.
Queries	GET /api/queries, /overview, /analytics, /advanced-analytics	History, overview, analytics, delete, pin, favorite, tags, and action tracking.
Payments	/current, /create-order, /create-payment-link, /verify, /verify-payment-link, /downgrade, /invoices	Plan checkout, verification, invoices, and downgrade.
Webhooks	POST /api/payment/webhook, POST /api/webhooks/clerk	Signed Razorpay and Clerk event processing.
Feedback	POST /api/feedback, GET /api/feedback/mine	User feedback submission and history.

Notifications	GET /api/notifications, PATCH /read-all, PATCH /:id/read	User notification inbox and read state.
Admin	/login, /me, /overview, /users, /feedback, /notifications, /access-appeals, /security-events	Admin metrics, moderation, publishing, triage, and review.

MONGODB AND MONGOOSE

Data Model Documentation

The models support identity, AI history, schema control, webhook processing, and security monitoring.

Model	Purpose
User	Identity mirror, role, status, access status, plan, billing renewal, usage, organization metadata, risk fields.
Schema	Saved schema context for personal or organization workspace.
Query	Prompt, generated SQL, mode, dialect, tags, pinned/favorite flags, copy/export counters.
Payment	Checkout, Razorpay order/link/payment IDs, amount, currency, status, scope, and plan.
Invoice	Invoice number, payment/order IDs, amount, currency, status, scope, and plan.
OrganizationSubscription	Organization-level plan, billing provider, payment IDs, renewal window, and status.
Feedback	Rating, topic, message, status, admin note, workspace owner, and user reference.
Notification and NotificationRead	Announcements, audience targeting, priority, read tracking, status, publish/expiry dates.
AccessAppeal	User appeal records with email, message, status, admin note, risk context, and metadata.

SecurityEvent	Security signals, severity, status, risk impact, user snapshot, metadata, IP, and user agent.
AdminAuditLog	Admin action, reason, target user snapshot, previous and next state, request metadata.
WebhookAuditLog	Provider event ID/type, Clerk identity, status, payload summary, and failure notes.

CONTROLS ALREADY IMPLEMENTED

Security and Quality

- Clerk sessions are the default user auth model; legacy user JWTs stay disabled unless explicitly enabled.
- Admin password login issues an HttpOnly cookie; the browser no longer needs to store an admin JWT in localStorage.
- CORS is allowlist-based and CSRF checks reject unsafe browser mutations from untrusted origins.
- Helmet, route-specific rate limits, request body limits, express-validator rules, and centralized error responses reduce API risk.
- Razorpay payment verification and webhooks use HMAC signatures and local ownership checks.
- Production env validation rejects placeholder secrets, weak secrets, HTTP public origins, and partial integration secrets.
- Workspace scoping keeps personal and organization data separated across schema, query, invoice, feedback, and analytics flows.

Validation and resilience

Input validation

express-validator rules cover AI requests, schema size, query history filters, IDs, feedback, admin queries/actions, notification creation/status, and payment payloads.

Rate limiting

Separate limiters protect auth, admin login, AI requests, payment requests, payment webhooks, and password-reset-style flows.

Request sizing

Dedicated body limits keep tiny payment calls, standard JSON, schema JSON, AI JSON, and raw webhook payloads bounded.

Logging

Backend and frontend loggers redact sensitive keys such as passwords, tokens, secrets, cookies, API keys, authorization headers, and signatures.

IMPLEMENTATION MAP

Technology and Project Structure

Area	Implementation
------	----------------

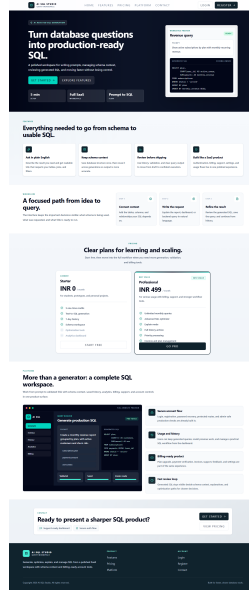
Frontend	React 19, Vite, React Router, Tailwind CSS v4, Clerk React, Lucide, Framer Motion, Recharts.
Backend	Node.js, Express 5, MongoDB, Mongoose, Clerk Express, Gemini client, Razorpay, Nodemailer, PDFKit.
Quality	Vitest, Testing Library, Node test runner, Supertest, ESLint, OpenAPI JSON, structured validators.
Deployment	Frontend/backend Dockerfiles, nginx config, docker-compose.yml, render.yaml, environment examples.

Important source locations

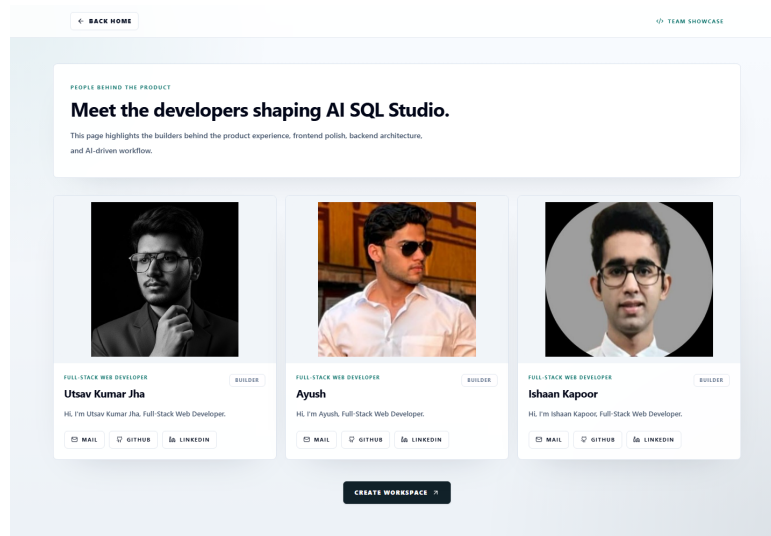
- Frontend routes: Frontend_Part/src/routes/AppRoutes.jsx
- AI workspace UI: Frontend_Part/src/pages/dashboard/Generate.jsx
- AI backend service: Backend_Part/src/services/ai.service.js
- Schema context: Frontend_Part/src/pages/dashboard/Schema.jsx and Backend_Part/src/services/schema.service.js
- Query history and analytics: Frontend_Part/src/pages/dashboard/History.jsx, Analytics.jsx, and Backend_Part/src/services/query.service.js
- Billing: Frontend_Part/src/pages/dashboard/Pricing.jsx, BillingSuccess.jsx, Invoices.jsx, and Backend_Part/src/services/payment.service.js
- Admin console: Frontend_Part/src/pages/admin/AdminDashboard.jsx and Backend_Part/src/services/admin.service.js
- API contract: Backend_Part/src/docs/openapi.js
- Deployment: Dockerfiles, nginx.conf, docker-compose.yml, render.yaml

VISUAL SHOWCASE

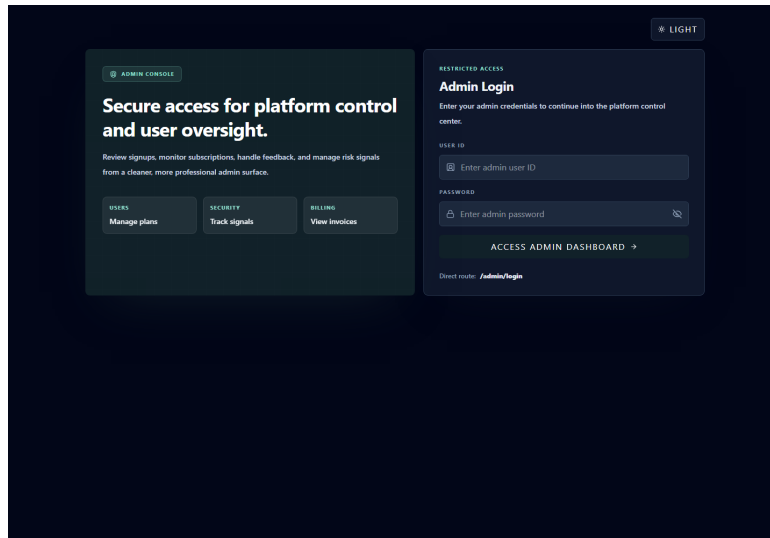
Screenshots



Public website screenshot showing the full landing-page experience.



Developers showcase page.



Separate admin login page for the platform console.

PROJECT QUALITY

Testing, Verification, and Deployment

Existing quality signals

- Backend tests cover API validation, auth guards, OpenAPI docs, indexes, env validation, Razorpay webhook security, and payment ownership checks.
- Frontend tests cover protected routing, AI service timeout behavior, query service URL building, HTTP auth events, and bodyless request headers.
- Frontend ESLint config enforces React hooks and refresh-safe exports.
- Docker and Render config make local and hosted deployment straightforward.
- OpenAPI document provides a public API contract for consumers and reviewers.

Professional presentation notes

- This PDF intentionally documents both product features and implementation evidence so it can be used in portfolio reviews, project submissions, or stakeholder demos.
- The website demonstrates frontend engineering, backend architecture, AI integration, auth, billing, analytics, admin operations, security hardening, and deployment readiness.
- The remaining polish opportunities are mostly product expansion items, not missing core architecture: richer live screenshots, more OpenAPI schemas, automated screenshot capture in CI, and deeper integration tests with a test database.

SUMMARY

Final Product Pitch

AI SQL Studio in one sentence

A professional full-stack SaaS website that turns schema-aware prompts into useful SQL and wraps that core AI workflow with authentication, plans, billing, analytics, history, support, notifications, admin operations, security controls, tests, and deployment configuration.

For a portfolio or academic review, the strongest story is that this is not just an AI feature. It is a complete web product: public acquisition pages, protected workspace, paid-plan lifecycle, admin governance, and production-style backend architecture all exist in the repository.